OS Simulator

A console-based operating system simulator in pure Java — project walkthrough and sample output

About this document

This walkthrough shows the OS Simulator running end to end. The program models the core decisions a real operating system makes across three classic topics: CPU scheduling, memory page replacement, and deadlock avoidance. Each section below shows real terminal output captured from the running program, with a short note on what it demonstrates.

What it implements

CPU Scheduling: FCFS, SJF (non-preemptive), and Round Robin (preemptive), each with a Gantt chart and per-process completion, turnaround, and waiting times.
Memory — Page Replacement: FIFO, LRU, and Optimal, run on the same reference string and compared by page faults and hit ratio.
Deadlock — Banker's Algorithm: computes the Need matrix, checks for a safe state, and reports a safe sequence.

Tech: Java (JDK 21), standard library only. Console / menu-driven. Source: github.com/deepthishivani/OS-Simulator

1. Main Menu & Adding Processes

The simulator is menu-driven. The user adds processes (each with an arrival time, burst time, and priority), then chooses an algorithm to run. Below, three processes are added: P1 (arrival 0, burst 5), P2 (arrival 1, burst 4), and P3 (arrival 2, burst 2).

```
deepthishivani@Deepthis-MacBook-Air OSSimulator % javac *.java
java OSSimulator

===== OS SIMULATOR =====
1. Add a process
2. View all processes
3. Run FCFS Scheduling
4. Run SJF Scheduling
5. Run Round Robin Scheduling
6. Run Page Replacement (Memory)
7. Run Banker's Algorithm (Deadlock)
8. Exit
Choose an option: 1
Enter Process ID (number): 1
Enter Arrival Time: 0
Enter Burst Time: 5
Enter Priority: 1
Process P1 added successfully.

===== OS SIMULATOR =====
1. Add a process
2. View all processes
3. Run FCFS Scheduling
4. Run SJF Scheduling
5. Run Round Robin Scheduling
6. Run Page Replacement (Memory)
7. Run Banker's Algorithm (Deadlock)
8. Exit
Choose an option: 1
Enter Process ID (number): 2
Enter Arrival Time: 1
Enter Burst Time: 4
Enter Priority: 2
Process P2 added successfully.

===== OS SIMULATOR =====
1. Add a process
2. View all processes
3. Run FCFS Scheduling
4. Run SJF Scheduling
5. Run Round Robin Scheduling
6. Run Page Replacement (Memory)
7. Run Banker's Algorithm (Deadlock)
8. Exit
Choose an option: 1
Enter Process ID (number): 3
Enter Arrival Time: 2
Enter Burst Time: 2
Enter Priority: 3
Process P3 added successfully.
```

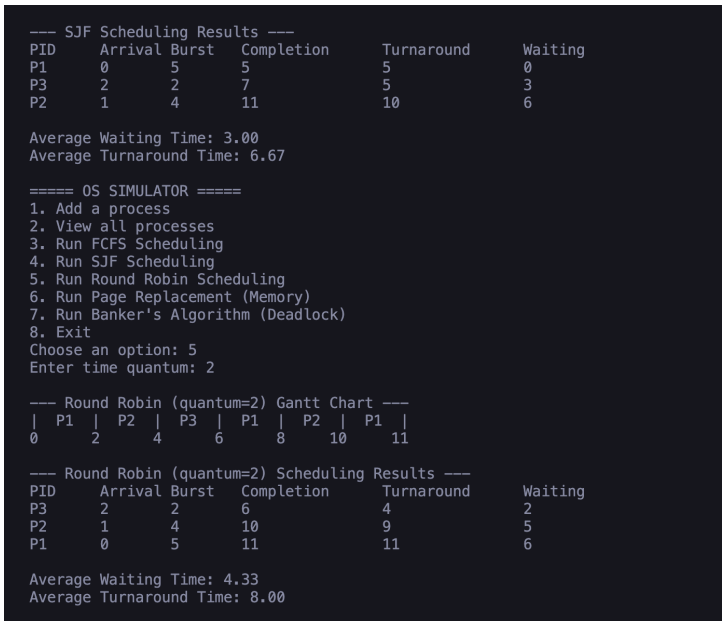
2. CPU Scheduling — FCFS and SJF

FCFS runs processes in arrival order (P1 → P2 → P3), giving an average waiting time of 3.67. Notice P3 waits 7 units despite needing only 2 — a short job stuck behind longer ones (the convoy effect). SJF then reorders to run the shortest available job first (P1 → P3 → P2), lowering the average waiting time to 3.00.

```
--- SJF Scheduling Results ---
PID     Arrival Burst   Completion      Turnaround      Waiting
P1      0       5       5               5               0
P3      2       2       7               5               3
P2      1       4       11              10              6

Average Waiting Time: 3.00
Average Turnaround Time: 6.67

===== OS SIMULATOR =====
1. Add a process
2. View all processes
3. Run FCFS Scheduling
4. Run SJF Scheduling
5. Run Round Robin Scheduling
6. Run Page Replacement (Memory)
7. Run Banker's Algorithm (Deadlock)
8. Exit
Choose an option: 5
Enter time quantum: 2

--- Round Robin (quantum=2) Gantt Chart ---
|  P1  |  P2  |  P3  |  P1  |  P2  |  P1  |
0      2      4      6      8      10     11

--- Round Robin (quantum=2) Scheduling Results ---
PID     Arrival Burst   Completion      Turnaround      Waiting
P3      2       2       6               4               2
P2      1       4       10              9               5
P1      0       5       11              11              6

Average Waiting Time: 4.33
Average Turnaround Time: 8.00
```

3. CPU Scheduling — Round Robin (Preemptive)

Round Robin gives each process a fixed time quantum (here, 2 units) before moving to the next. The Gantt chart shows the time-slicing directly: P1 appears three separate times because it is repeatedly preempted and cycled to the back of the queue. No process monopolises the CPU — the trade-off is fairness in exchange for a higher average waiting time (4.33) than SJF.



```
--- SJF Scheduling Results ---
PID     Arrival Burst   Completion      Turnaround      Waiting
P1      0       5       5               5               0
P3      2       2       7               5               3
P2      1       4       11              10              6

Average Waiting Time: 3.00
Average Turnaround Time: 6.67

===== OS SIMULATOR =====
1. Add a process
2. View all processes
3. Run FCFS Scheduling
4. Run SJF Scheduling
5. Run Round Robin Scheduling
6. Run Page Replacement (Memory)
7. Run Banker's Algorithm (Deadlock)
8. Exit
Choose an option: 5
Enter time quantum: 2

--- Round Robin (quantum=2) Gantt Chart ---
|  P1  |  P2  |  P3  |  P1  |  P2  |  P1  |
0      2      4      6      8     10     11

--- Round Robin (quantum=2) Scheduling Results ---
PID     Arrival Burst   Completion      Turnaround      Waiting
P3      2       2       6               4               2
P2      1       4       10              9               5
P1      0       5       11              11              6

Average Waiting Time: 4.33
Average Turnaround Time: 8.00
```

The Round Robin results table (P3, P2, P1 in finish order) and its averages appear directly below the Gantt chart in the same run.

4. Memory — Page Replacement (FIFO)

The user enters a number of frames and a reference string. Each algorithm processes the same string and shows, step by step, the frame contents and whether each access was a hit or a page fault. FIFO evicts the oldest page in memory and produces 10 page faults here.

```
===== OS SIMULATOR =====
1. Add a process
2. View all processes
3. Run FCFS Scheduling
4. Run SJF Scheduling
5. Run Round Robin Scheduling
6. Run Page Replacement (Memory)
7. Run Banker's Algorithm (Deadlock)
8. Exit
Choose an option: 6
Enter number of frames: 3
How many page references? 13
Enter the 13 page numbers one by one:
7 0 1 2 0 3 0 4 2 3 0 3 2

Reference string: 7 0 1 2 0 3 0 4 2 3 0 3 2
Number of frames: 3

--- FIFO ---
Page    Frames          Status
7       [7]             Fault
0       [7, 0]          Fault
1       [7, 0, 1]               Fault
2       [0, 1, 2]               Fault
0       [0, 1, 2]               Hit
3       [1, 2, 3]               Fault
0       [2, 3, 0]               Fault
4       [3, 0, 4]               Fault
2       [0, 4, 2]               Fault
3       [4, 2, 3]               Fault
0       [2, 3, 0]               Fault
3       [2, 3, 0]               Hit
2       [2, 3, 0]               Hit
FIFO total page faults: 10
```

5. Memory — LRU and Optimal

LRU evicts the page unused for the longest time (9 faults). Optimal evicts the page not needed for the longest time into the future (7 faults) — the theoretical best, since it relies on knowing future requests.

```
--- LRU ---
Page    Frames          Status
7       [7]             Fault
0       [7, 0]          Fault
1       [7, 0, 1]               Fault
2       [0, 1, 2]               Fault
0       [1, 2, 0]               Hit
3       [2, 0, 3]               Fault
0       [2, 3, 0]               Hit
4       [3, 0, 4]               Fault
2       [0, 4, 2]               Fault
3       [4, 2, 3]               Fault
0       [2, 3, 0]               Fault
3       [2, 0, 3]               Hit
2       [0, 3, 2]               Hit
LRU total page faults: 9

--- Optimal ---
Page    Frames          Status
7       [7]             Fault
0       [7, 0]          Fault
1       [7, 0, 1]               Fault
2       [2, 0, 1]               Fault
0       [2, 0, 1]               Hit
3       [2, 0, 3]               Fault
0       [2, 0, 3]               Hit
4       [2, 4, 3]               Fault
2       [2, 4, 3]               Hit
3       [2, 4, 3]               Hit
0       [2, 0, 3]               Fault
3       [2, 0, 3]               Hit
2       [2, 0, 3]               Hit
Optimal total page faults: 7
```

6. Memory — Comparison

All three algorithms are compared side by side on the same input. The ordering Optimal < LRU < FIFO (in page faults) is the expected result: Optimal sets the performance ceiling, LRU is the realistic approximation, and FIFO is simplest but least efficient.

```
===== COMPARISON =====
Algorithm  PageFaults   Hits     HitRatio
FIFO       10           3        0.23
LRU        9            4        0.31
Optimal    7            6        0.46

Fewer faults = better. Optimal is the theoretical best (it can see the future).
```

7. Deadlock Avoidance — Banker's Algorithm

Given the Allocation matrix, Max matrix, and Available resources, the program computes Need = Max − Allocation, then searches for a safe sequence. A process can run if its entire Need row fits within the currently available resources; when it finishes it returns its resources, freeing them for the next. Here the system is SAFE, with safe sequence P1 → P3 → P4 → P0 → P2.

```
===== OS SIMULATOR =====
1. Add a process
2. View all processes
3. Run FCFS Scheduling
4. Run SJF Scheduling
5. Run Round Robin Scheduling
6. Run Page Replacement (Memory)
7. Run Banker's Algorithm (Deadlock)
8. Exit
Choose an option: 7
Enter number of processes: 5
Enter number of resource types: 3
Enter the Allocation matrix (5 rows x 3 values):
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter the Max matrix (5 rows x 3 values):
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter the Available resources (3 values):
3 3 2

Need (Max - Allocation):
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1

System is in a SAFE state.
Safe sequence: P1 -> P3 -> P4 -> P0 -> P2
```